

A Traffic-Aware Adaptive Intrusion Detection System for Dynamic Network Conditions

¹Satish Dekka, ²P. Pavani, ³S. Kamal Kumar, ⁴V. Jahnavi, ⁵S. Sirisha

^{1,2,3,4,5}Department of Computer Science and Engineering, Lendi Institute of Engineering and Technology, Vizianagaram, India

Abstract—The Intrusion Detection System is an indispensable component in ensuring security in computer networks. At present, due to the rapidly growing complexity of networks, it becomes challenging to detect any kind of intrusion accurately. Most intrusion detection systems rely on using a single algorithmic approach to differentiate between normal and suspicious network activities. Despite achieving satisfactory results under some conditions, such an approach loses its effectiveness in different situations caused by varying network traffic conditions. Cases of predominant normal traffic, sudden attacks, or complex network conditions can reduce the accuracy of such a method and raise the number of false positives. It involves using multiple machine learning and deep learning approaches, including decision trees, random forests, convolutional neural networks (CNNs), long short-term memory (LSTM), and hybrid CNN-LSTMs. Common intrusion detection datasets will be used for analysis in order to measure the effectiveness of each methodical approach and replicate various network traffic conditions. The experimental results demonstrate that the adaptive method enhances detection accuracy, reduces the false alarm rate, and fortifies the overall robustness of the system in comparison to traditional static intrusion detection systems. This problem can be solved well by using the adaptive model.

Key Words— Intrusion Detection Systems, Network Security, Adaptive Detection, Machine Learning, Deep Learning, Convolutional Neural Networks, Long Short-Term Memory.

I. INTRODUCTION

Technology advancements and networking applications have made significant progress in the field of computer networks. Computer networks help make a lot of useful

services possible, such as online banking, cloud computing, e-commerce, communication, and social media. This change has made it easy for people all over the world to talk to each other and get information from anywhere in the world. But as more people use computer networks, new problems arise, especially when it comes to keeping the networks safe. A lot of cybercriminals keep trying to use the flaws in networking systems to attack computer networks. There are many kinds of network attacks, such as denial-of-service attacks, probing attacks, remote-to-local attacks, and user-to-root attacks. This could lead to a lot of big problems and losses, including a data breach and the network's service being interrupted. So, it's important to put some measures in place to keep an eye on network traffic so that you can spot any suspicious activity right away. The intrusion detection systems are vital in ensuring network security through the analysis of the network traffic and identification of any possible attacks. Intrusion detection system techniques are usually categorized as signature-based detections for known attacks and anomaly-based detections for abnormal behavior. Algorithms that are commonly used in this case include decision tree and random forest approaches [19]. Also, deep learning architectures like CNN and LSTM have shown that they can recognize complex spatial-temporal patterns in network traffic patterns very well [7,10]. Combining the strengths of both deep learning and traditional methods, such as CNN-LSTM, can improve detection even more. After each model undergoes training, it will be tested using performance metrics like accuracy, precision, recall, and false positive rate. Given the nature of the traffic, the adaptive model selection process will pick out the best model for detecting intrusions. The benefit of this approach is that it increases the efficiency of the intrusion detection system.

1.1 Research Gap

Many intrusion detection systems have already been developed using machine learning and deep learning methods. However, most of them still depend on a single model to classify network traffic. These models may work well in controlled or ideal conditions, but they do not always give the same performance when network traffic keeps changing. In real networks, traffic is not fixed. It keeps changing because of different users, system load, and possible cyber-attacks. Because of this, using only one model is not always reliable. So, there is a need for a system that can adjust itself based on the current traffic and choose the most suitable model for better detection.

1.2 Objectives of the Study

The main objectives of this study are:

1. To design a system that can detect if the network traffic is normal or if there is any attack.
2. To use different machine learning and deep learning models and see how they work for detecting intrusions.
3. To study how the network traffic changes under different conditions during testing.
4. To compare the models using metrics like accuracy, precision, recall and false positive rate.
5. To make a system that can select the best model based on the current traffic condition.

1.3 Main Contributions

1. In this project, a traffic-aware intrusion detection system is developed which can change its working based on the network traffic.
2. Different machine learning and deep learning models such as Decision Tree, Random Forest, CNN, LSTM and Hybrid CNN-LSTM are used for detecting intrusions.

3. An adaptive method is also used where the system selects the suitable model depending on the traffic condition at that time.
4. The system is tested using the NSL-KDD dataset, and the performance of models is observed using different evaluation metrics.
5. The main goal is to reduce false alarms and improve the overall reliability of the system in different network situations.

1.4 Organization of the Paper

The rest of the paper is organized in the following way. Section II gives an overview of the literature review, where some of the earlier works related to intrusion detection and machine learning-based security are discussed. In Section III, the proposed methodology is explained, which includes details about the system, dataset, preprocessing steps, models used, and the adaptive selection approach. Section IV shows the results and discussion part, where the performance of different models is analyzed using the NSL-KDD dataset. Section V provides the conclusion by summarizing the important outcomes of the project. Section VI briefly talks about the future scope and possible improvements. Finally, Section VII contains the references used in this work.

II. LITERATURE REVIEW

Network intrusion detection is really important these days. This is because cyber-attacks are happening often and network systems are getting more complicated. So a lot of researchers are trying to use machine learning and deep learning to make intrusion detection systems. The main goal is to make these systems good at handling network traffic that is always changing, finding attacks that are not known and working well with huge amounts of data. Some researchers are also looking into deep learning models. For instance Liu came up with a model that

uses a Transformer and an Adversarial Autoencoder together. In this model the Transformer looks at network traffic patterns over time. The Adversarial Autoencoder helps stop the system from getting too good at one thing and improves detection of new network intrusion detection attacks. Network intrusion detection systems like this were tested on datasets like KDD99 and CICIDS2017. The results showed that this model works better than network intrusion detection methods especially when it comes to being accurate and not sending out false alarms, about network intrusion detection. Similarly, Villegas-Ch et al. developed an adaptive deep learning-based intrusion detection system designed to improve threat detection in dynamic cybersecurity environments. Their system continuously learns from network traffic data and adapts to evolving cyber threats. Experimental results demonstrated significant improvements in detection accuracy, precision, recall, and response time when compared to traditional rule-based intrusion detection approaches. Machine learning techniques have also been widely applied in intrusion detection systems. Thaseen and Kumar developed a machine learning-based IDS using algorithms such as Naïve Bayes, Decision Tree, and Support Vector Machine for

detecting malicious network activities. Their experimental analysis showed that machine learning classifiers can effectively improve intrusion detection performance. Cantone et al. conducted a cross-dataset generalization study to analyze the robustness of machine learning-based intrusion detection models. Their research evaluated multiple classifiers across different intrusion detection datasets and found that although models perform well on the dataset used for training, their performance decreases significantly when applied to unseen datasets. The study emphasized the importance of designing IDS models capable of generalizing across different network environments. Deep learning models have demonstrated promising performance in identifying complex network attack patterns. Du et al. proposed a CNN-LSTM based intrusion detection model known as NIDS-CNN-LSTM for Industrial Internet of Things environments. The model combines convolutional neural networks for spatial feature extraction with long short-term memory networks to capture temporal traffic patterns. Experimental results on KDD Cup 99, NSL-KDD, and UNSW-NB15 datasets showed improved classification accuracy and reduced false alarm rates compared to conventional IDS methods [4], [11].

2.1 Summary of Literature Review

No	Au- thor(s)	Year	Model	Dataset	Key Contribu- tion	Limitation
1	Weiwei Liu	2024	Transformer + Ad- versarial Autoen- coder	KDD99, CICIDS2017	Improved detec- tion accuracy us- ing self-attention and feature opti- mization.	High computa- tional complexity for real-time sys- tems.
2	Ville- gas-Ch et al.	2024	Adaptive Deep Learning IDS	Standard IDS datasets	Adaptive IDS ca- pable of learning evolving cyber threats.	Requires continu- ous training and high computa- tional resources.

3	Shahid et al.	2024	Hybrid ML/DL IDS	ROUT-4-2023	Achieved high accuracy for IoT network attack detection.	Focused mainly on IoT routing attacks.
4	Du et al.	2023	CNN-LSTM	KDD99, NSL-KDD, UNSW-NB15	Extracts spatial and temporal features for IIoT intrusion detection.	High training time and computational overhead.
5	Altunay & Albayrak	2023	CNN + LSTM Hybrid	UNSW-NB15, X-IIoTID	Improved detection accuracy in Industrial IoT environments.	Model complexity increases system cost.

III. PROPOSED METHODOLOGY

In this project, a Traffic-Aware Adaptive Intrusion Detection System is proposed to improve how intrusions are detected in changing network environments. Usually, traditional intrusion detection systems depend on a single model, and this may not work well when network traffic keeps changing. To solve this problem, the system uses multiple machine learning and deep learning algorithms and then selects the most suitable one based on the current network traffic condition.

The proposed system is built in multiple steps, starting from dataset preparation and data preprocessing, followed by model training and traffic analysis. After that, an adaptive selection process is used to choose the most suitable model. Finally, the system performs the intrusion detection. It continuously observes the incoming network traffic from the various sources and it uses the selected model to decide whether the activity is normal or malicious.

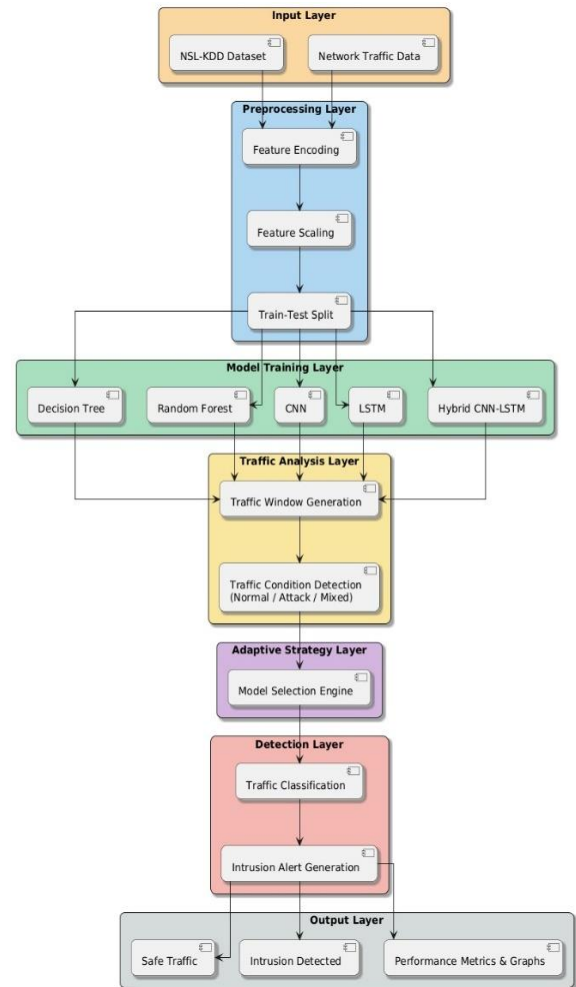


Fig 3.1 System Architecture of Traffic-Aware Adaptive IDS

A. System Overview

In the proposed system, the process begins with collecting network traffic data. After that, the data is cleaned and important features are taken for further processing. Then, different models are trained and their performance is checked. Based on the network condition, the system chooses the model that works best at that moment. Finally, it identifies whether the traffic is normal or malicious and generates an alert if any intrusion is detected. First, the system uses network traffic data from the NSL-KDD dataset, which includes both normal activities and different types of attacks. The data is then cleaned and prepared so it can be used properly for further processing. After that, the important features are taken and used to train different intrusion detection models. Once the training is done, the models are tested to see how well they perform using measures like accuracy and confusion matrix. Then, depending on the current network traffic, the system decides which model is more suitable at that moment. In the final stage, it checks the incoming traffic, classifies it as normal or malicious, and generates an alert if any attack is detected.

B. Dataset Description

The proposed system uses the NSL-KDD dataset, which is a widely used benchmark dataset for evaluating intrusion detection systems. The dataset is an improved version of the KDD Cup 1999 dataset and addresses several issues such as redundant records and biased training data. Each record in the NSL-KDD dataset contains 41 features that describe various characteristics of network connections. These features include basic network traffic attributes, content features, and traffic behavior statistics. The dataset includes both normal network traffic and multiple attack categories, including:

1. Denial of Service (DoS)
2. Probe attacks
3. Remote-to-Local (R2L) attacks
4. User-to-Root (U2R) attacks

Using this dataset allows the proposed system to evaluate the effectiveness of intrusion detection models under different attack scenarios.

C. Data Preprocessing

Before using the dataset, some preprocessing steps are needed so that it works properly with machine learning and deep learning models. The NSL-KDD dataset has both numerical and categorical values, so it cannot be used directly. First, the data is cleaned by removing duplicate and unnecessary records. This helps in reducing noise and makes the dataset more reliable for further processing. After the cleaning step, the categorical values like protocol type, service and flag cannot be used directly, so they are converted into numbers using label encoding. This is required because the models only understand numerical data. Then, scaling is done so that all the feature values stay in a similar range, otherwise some values may affect the model more than others. In the last step, the dataset is divided into two parts. One part is used for training the models and the other part is used to test how well the models are performing.

D. Feature Extraction and Traffic Pattern Analysis

After preprocessing, relevant features are extracted from the dataset to analyze traffic pattern. The Feature extraction helps identify important attributes that contribute to distinguishing between normal and malicious network traffic. Traffic pattern analysis is then performed to understand the characteristics of network behavior. By analyzing these patterns, the system can identify abnormal activities that may indicate potential cyber-attacks. This step improves the effectiveness of machine learning models by providing meaningful features for training.

E. Intrusion Detection Models

The proposed system evaluates five different intrusion detection algorithms:

1. Decision Tree (DT)

Decision Tree is a supervised machine learning algorithm that classifies data by creating a tree-like structure of decision rules. It is easy to interpret and provides fast classification for network traffic data. The following formulas are:

Entropy:

$$Entropy(S) = - \sum_{i=1}^n p_i \log_2(p_i)$$

Information Gain:

$$IG(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

2.Random Forest (RF)

Random Forest is an ensemble learning method that combines multiple decision trees to improve classification accuracy and reduce overfitting. It is widely used in intrusion detection because of its robustness and high performance.

$$H(x) = mode\{h_1(x), h_2(x), \dots, h_n(x)\}$$

3.Convolutional Neural Network (CNN)

CNN is a deep learning model capable of extracting important features from input data. In intrusion detection systems, CNN helps identify spatial patterns within network traffic features.

Convolution operation:

$$F(i, j) = \sum_m \sum_n X(i - m, j - n) K(m, n)$$

Activation function:

$$ReLU(x) = \max(0, x)$$

4.Long Short-Term Memory (LSTM)

LSTM is a type of recurrent neural network designed to capture sequential relationships in time-series data. It is useful for analyzing network traffic sequences and detecting temporal attack patterns.

Forget gate:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Input gate:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Cell state update:

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t$$

5.Hybrid CNN-LSTM Model

The hybrid CNN-LSTM model combines the feature extraction capability of CNN with the sequential learning ability of LSTM. This hybrid architecture enables the system to detect complex attack patterns more effectively than individual models.

CNN extracts spatial features:

$$Z = CNN(X)$$

LSTM captures temporal dependencies:

$$Y = LSTM(Z)$$

Final classification:

$$\hat{y} = Softmax(WY + b)$$

F. Model Evaluation

After training, the performance of each intrusion detection model is evaluated using testing data. Evaluation metrics such as accuracy and confusion matrix are used to measure classification performance.

G. Dynamic Network Condition Analysis and Adaptive Model Selection

The key contribution of the proposed system is the adaptive strategy selection mechanism. Instead of using a single detection model, the system evaluates the performance of all trained models under different traffic conditions.

The system analyzes the characteristics of incoming network traffic and identifies the current traffic condition, such as normal-dominant traffic, attack-dominant traffic, or mixed traffic scenarios. Based on this analysis, the adaptive strategy module selects the model that performs best for the detected traffic condition.

This dynamic model selection approach allows the intrusion detection system to maintain high detection accuracy and reduce false alarms in dynamic network environments.

H. Intrusion Detection and Alert Generation

After selecting the suitable model for the task, the selected model is applied to identify network traffic. Network traffic is evaluated, and a decision on whether it is legitimate or malicious traffic is made. In case of an intruder, an alarm is set off, and the network administrator is notified of such an event. This alert mechanism helps administrators quickly respond to potential threats and take appropriate security measures to protect the network.

I. Performance Evaluation

For assessing the efficiency of the proposed scheme, several parameters have been considered. These include accuracy, precision, recall, and F1-score. All these parameters assess the performance of the intrusion detection system for distinguishing between the regular and intrusive network data packets. Through such assessment, the adaptive scheme can be able to select the most efficient intrusion detection approach.

J. Experimental Setup

The experimental work was done in Python, which is an efficient programming language for machine learning and deep learning. Machine learning algorithms such as decision tree and random forest were implemented with the help of Scikit-learn, whereas CNN, LSTM, and hybrid CNN-LSTM were implemented using TensorFlow and Keras. The datasets NSL-KDD were used to train and test the models. For the NSL-KDD datasets, a 70/30 training/testing partition ratio was used to divide the data. The performance of the models was calculated based on accuracy, precision, recall, F1-score, and confusion matrix.

ALGORITHMS

Algorithm 1: Decision Tree Based Intrusion Detection

Input: Preprocessed network traffic dataset X , labels Y

Output: Predicted class labels and evaluation metrics.

Steps:

1: Load NSL-KDD dataset

2: Perform data preprocessing

- a. Remove irrelevant attributes
- b. Encode categorical features
- c. Apply feature scaling

3: Split dataset into training set and testing set

4: Initialize Decision Tree classifier

5: Train Decision Tree model using training dataset

6: For each instance x_i in testing dataset do

7: Traverse decision nodes based on feature conditions

8: Assign class label (Normal or Attack) at leaf node

9: End For

10: Compute evaluation metrics

- a. Accuracy
- b. Confusion Matrix

11: Output predicted class labels

Algorithm 2: Random Forest Based Intrusion Detection

Input: Training Dataset, Testing Dataset

Output: Predicted class labels and evaluation metrics

Steps:

1: Load NSL-KDD dataset

2: Perform preprocessing and feature scaling

3: Split dataset into training and testing sets

4: Initialize Random Forest with N decision trees

5: For each tree T_i in the forest do

6: Select random bootstrap sample from training data

7: Select random subset of features

8: Train decision tree T_i

```

9: End For
10: For each test sample  $x_j$  do
11: Obtain prediction from each tree
12: Apply majority voting
13: Assign final class label
14: End For
15: Compute model performance using
    a. Accuracy
    b. Confusion Matrix
16: Output intrusion detection result

```

Algorithm 3: CNN Based Intrusion Detection

Input: Preprocessed network traffic features X

Output: Binary classification (Normal / Attack)

Steps:

```

1: Load NSL-KDD dataset
2: Perform preprocessing and normalization
3: Encode categorical attributes
4: Split dataset into training and testing sets
5: Reshape feature matrix for CNN input
6: Construct CNN architecture
    a. Input layer
    b. Convolution layer
    c. Max pooling layer
    d. Fully connected layer
7: Train CNN model using training dataset
8: For each test sample  $x_i$  do
9: Extract spatial features using convolution layers
10: Apply sigmoid activation for classification

```

```

11: End For
12: Compute evaluation metrics
    a. Accuracy
    b. Confusion Matrix
13: Output prediction results

```

Algorithm 4: LSTM Based Intrusion Detection

Input: Preprocessed network traffic dataset

Output: Detection result (Normal / Attack)

Steps:

```

1: Load NSL-KDD dataset
2: Perform data preprocessing
3: Normalize and scale input features
4: Split dataset into training and testing sets
5: Reshape dataset into sequential format
6: Construct LSTM architecture
    a. Input layer
    b. LSTM layer
    c. Dense output layer
7: Train LSTM model using training dataset
8: For each network sequence  $x_i$  do
9: Capture temporal patterns using LSTM memory cells
10: Predict class label (Normal / Attack)
11: End For
12: Evaluate model using
    a. Accuracy
    b. Confusion Matrix

```

13: Output detection results

Algorithm 5: Hybrid CNN-LSTM Based Intrusion Detection

Input: Preprocessed network traffic dataset

Output: Predicted class labels

Steps:

- 1: Load NSL-KDD dataset
- 2: Perform preprocessing and feature scaling
- 3: Split dataset into training and testing sets
- 4: Reshape input data for hybrid model
- 5: Construct Hybrid CNN-LSTM architecture
 - a. CNN layers for spatial feature extraction
 - b. LSTM layer for temporal sequence learning
 - c. Fully connected classification layer
- 6: Train hybrid model using training dataset
- 7: For each test sample x_i do
- 8: Extract spatial features using CNN
- 9: Capture temporal patterns using LSTM
- 10: Predict traffic class
- 11: End For
- 12: Evaluate model performance using
 - a. Accuracy
 - b. Confusion Matrix
- 13: Output intrusion detection results.

Algorithm 6: Adaptive IDS Model Selection

Input: Network traffic condition

Output: Selected IDS model and detection result

Steps:

- 1: Monitor incoming network traffic
- 2: Analyze traffic characteristics
- 3: Determine network condition C
 - a. Normal traffic
 - b. Mixed traffic
 - c. Attack dominant traffic
- 4: If $C = \text{Normal}$ then
- 5: Select Decision Tree model
- 6: Else if $C = \text{Mixed}$ then
- 7: Select Random Forest model
- 8: Else
- 9: Select Hybrid CNN-LSTM model
- 10: End If
- 11: Apply selected model to detect intrusions
- 12: Classify traffic as Normal or Attack
- 13: Generate alert if attack is detected

IV. RESULTS AND DISCUSSIONS

In this part, the results of the experiments conducted on the implemented Traffic-Aware Adaptive Intrusion Detection System (IDS) are discussed. Various models such as machine learning and deep learning models were tested for their performance with the help of the NSL-KDD data set. The various models used in this study include Decision Tree, Random Forest, CNN, LSTM, and CNNT to measure the effectiveness of each model, several evaluation metrics were used, including Accuracy, Precision, Recall, and F1-Score. These metrics are derived from the confusion matrix and provide insight into how effectively each model detects malicious network traffic.

A. Model Evaluation Metrics

The performance of the intrusion detection models is evaluated using a confusion matrix consisting of four components:

1. True Positive (TP): Attack traffic correctly classified as attack
2. True Negative (TN): Normal traffic correctly classified as normal
3. False Positive (FP): Normal traffic incorrectly classified as attack
4. False Negative (FN): Attack traffic incorrectly classified as normal

Based on these values, the following evaluation metrics are calculated:

Accuracy

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Precision

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Recall

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

F1-Score

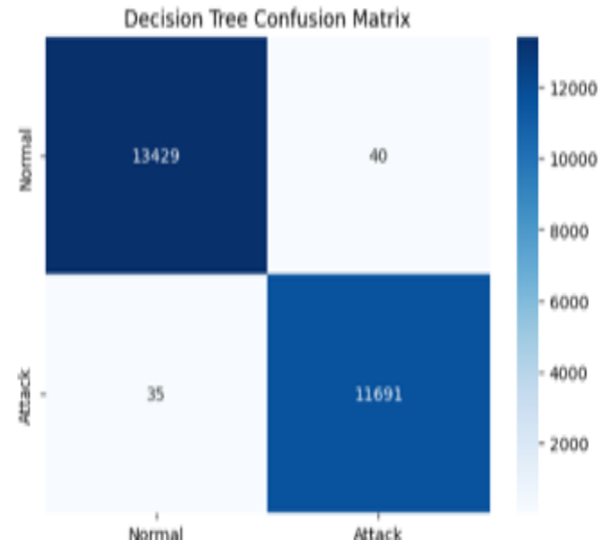
$$\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

B. Decision Tree Results

A decision tree model was tested using the NSL-KDD dataset, where the aim was to distinguish between two classes of network traffic (normal and attack). The decision tree model gave an accuracy rate of 99.70%, thus showing that it can effectively detect any malicious activity on the network. The precision, recall, and F1-scores for the two classes were all close to 1.00.

Confusion Matrix of Decision Tree Model

Actual / Predicted	Normal	Attack
Normal	13427	42
Attack	33	11693



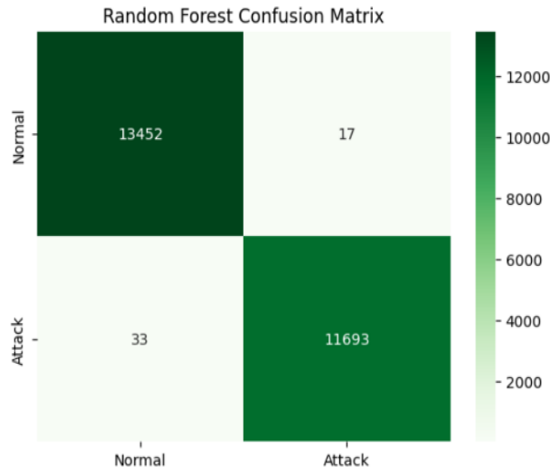
From the confusion matrix, it can be observed that 13,427 normal traffic instances and 11,693 attack instances were correctly classified. Only a small number of samples were misclassified, where 42 normal instances were incorrectly classified as attacks and 33 attack instances were incorrectly classified as normal.

These results indicate that the Decision Tree model achieves high detection accuracy with very low false positives and false negatives, making it an effective baseline model for network intrusion detection.

C. Random Forest Model Results

The Random Forest model was implemented to improve intrusion detection performance by utilizing an ensemble learning approach. Random Forest combines multiple decision trees and performs classification using majority voting, which helps reduce overfitting and improves model generalization.

The Random Forest model achieved an accuracy of 99.80% on the NSL-KDD testing dataset, demonstrating slightly improved performance compared to the Decision Tree model. The high accuracy indicates that the model effectively distinguishes between normal and malicious network traffic.



Confusion Matrix of Random Forest Model

Actual/Predicted	Normal	Attack
Normal	13452	17
Attack	33	11693

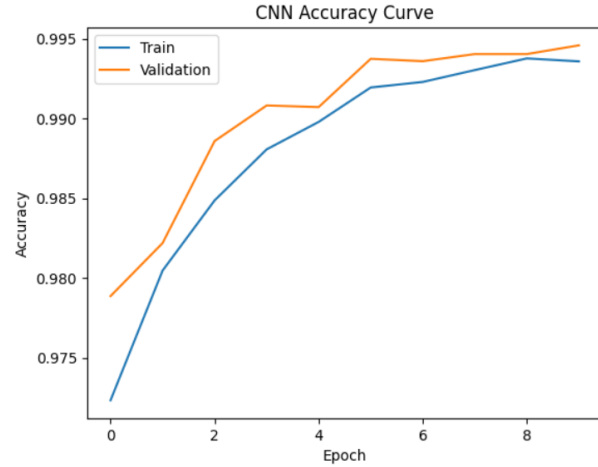
From the above confusion matrix, one can see that there were 13,452 correct classifications of normal traffic and 11,693 correct classifications of attack traffic. On the other hand, only 17 instances of normal traffic were wrongly classified as attack traffic, and 33 instances of attack traffic were wrongly classified as normal traffic.

The above analysis shows that Random Forest gives excellent results and is highly accurate in detecting intrusion compared to the Decision Tree algorithm since there are fewer misclassifications. In addition, it is an effective algorithm for solving high-dimensional problems due to its ensembling approach.

D. CNN Model Results

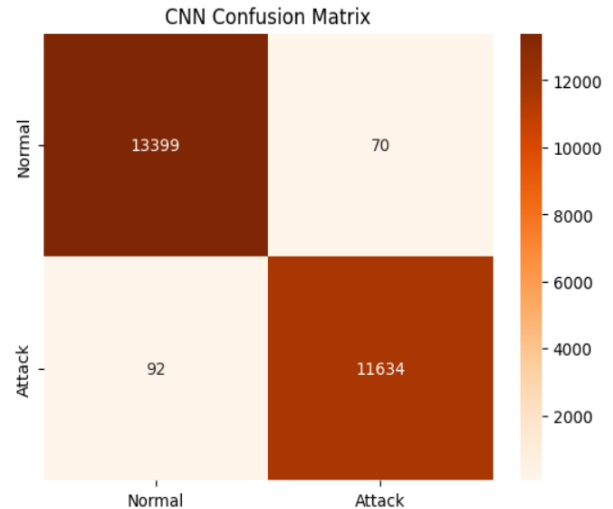
CNN algorithm is used because the purpose of using the CNN algorithm is to detect spatial relationship between different network traffic characteristics. CNN algorithms have the ability to extract complicated feature structures in high dimensional input space by performing convolution and pooling.

Training process: The CNN algorithm was trained for 10 epochs on the preprocessed NSL-KDD database. The training process resulted in gradual improvement



in the classifier's performance, whereby the accuracy increased from 97.23% in epoch one to about 99.36% in epoch ten. Also, there was an increase in the accuracy of the validation set up to 99.46%. The learning curve shown in Fig illustrates the training and validation accuracy across epochs, demonstrating that the model converges effectively without significant over-fitting.

Confusion Matrix of CNN Model



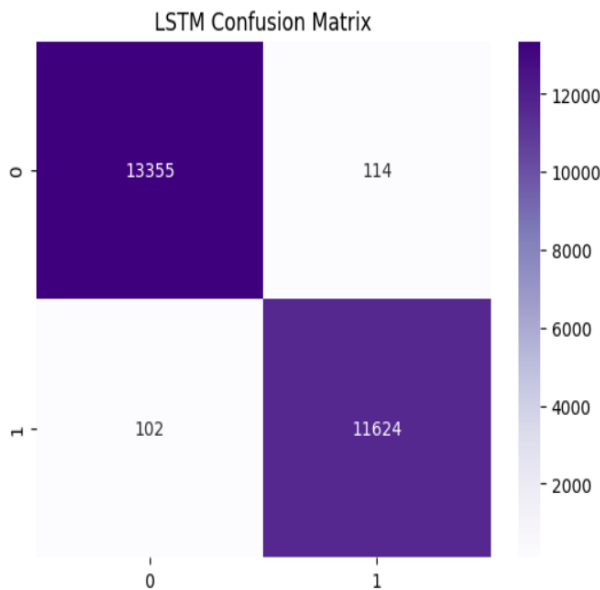
Actual/Predicted	Normal	Attack
Normal	13399	92
Attack	70	11634

From the confusion matrix, it can be observed that 13,399 normal traffic instances and 11,634 attack instances were correctly classified. A small number of samples were misclassified, where 70 normal instances were incorrectly predicted as attacks, and 92 attack instances were incorrectly classified as normal traffic. Although the CNN model achieved slightly lower accuracy compared to the Random Forest model, it demonstrates strong capability in detecting complex patterns in network traffic data. The results indicate that CNN can effectively learn feature representations and identify intrusion activities with high accuracy.

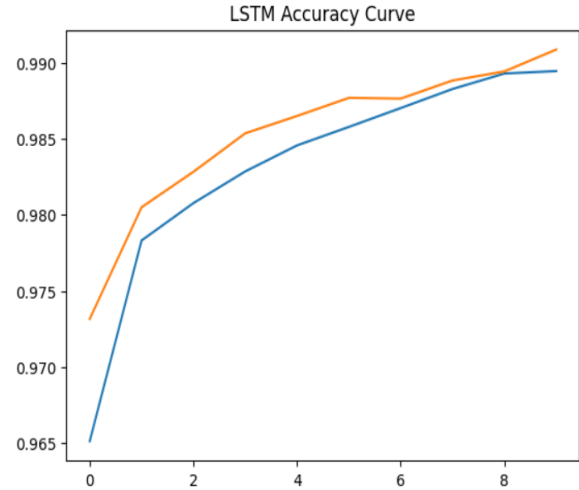
E. LSTM Model Results

The LSTM model was used to learn temporal dependencies and sequential behaviors in the network traffic data set. This is because LSTM networks are appropriate for intrusion detection as most cyber-attacks happen sequentially and depend on time. While training the LSTM model, the model was trained using 10 epochs on the preprocessed NSL-KDD dataset. There was an improvement in the training accuracy, which started at 96.51% at epoch one and ended at around 98.95% at epoch ten. Likewise, there was an increase in validation accuracy that reached around 99.09%. The curves indicate stable convergence with no significant signs of overfitting.

Confusion Matrix of LSTM Model



Actual/Predicted	Normal	Attack
Normal	13355	114
Attack	102	11624

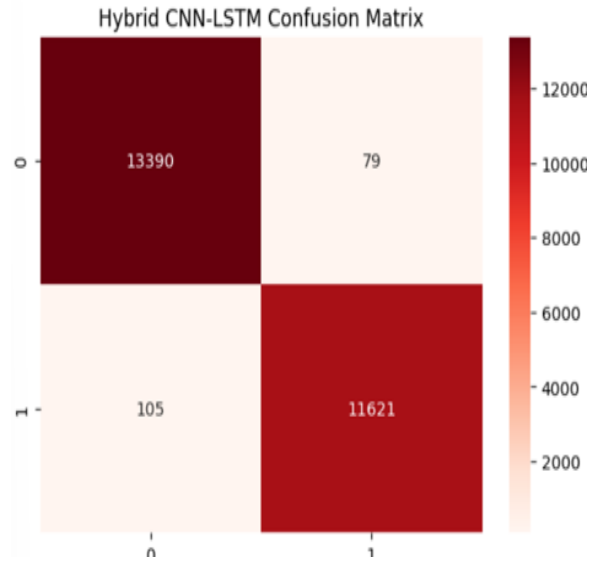


From the confusion matrix, it can be observed that 13,355 normal traffic instances and 11,624 attack instances were correctly classified by the LSTM model. A small number of samples were misclassified, where 114 normal instances were incorrectly predicted as attacks, and 102 attack instances were incorrectly classified as normal traffic. These results demonstrate that the LSTM model provides strong intrusion detection capability by learning temporal patterns in network traffic, making it suitable for detecting multi-stage and time-dependent cyber-attacks.

F. Hybrid CNN-LSTM Model Results

The Hybrid CNN-LSTM model was designed to utilize the benefits of both CNN and LSTM networks. In this model, the role of CNN is to capture the spatial features of network traffic data, whereas the LSTM layer handles the temporal aspect of the data. The Hybrid CNN-LSTM model allows detection of multi-staged cyber-attacks. In the course of training, the Hybrid CNN-LSTM model was trained for 10 epochs based on the NSL-KDD dataset that was preprocessed. With respect to accuracy, it was observed that there was steady growth from 94.80% in epoch 1 to about 99.10% in epoch 10. Finally, the result of evaluation with testing data yielded overall accuracy at around

99.27%. This is the highest among all the models that have been considered in this research work.



Confusion Matrix of Hybrid CNN–LSTM Model

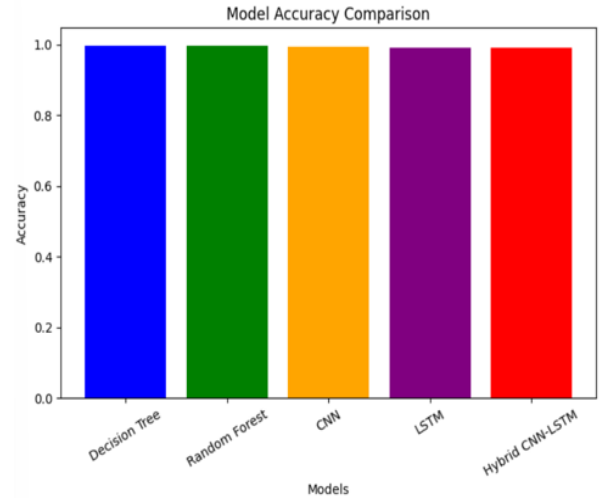
Actual/Predicted	Normal	Attack
Normal	13390	79
Attack	105	11621

From the confusion matrix, it can be observed that 13,390 normal traffic instances and 11,621 attack instances were correctly classified by the model. A small number of samples were misclassified, where 79 normal instances were incorrectly predicted as attacks, and 105 attack instances were incorrectly classified as normal traffic. These results demonstrate that the Hybrid CNN–LSTM model provides high intrusion detection accuracy with minimal misclassification, outperforming the individual CNN and LSTM models. By combining spatial and temporal feature learning, the hybrid model is capable of identifying complex intrusion patterns and dynamic attack behaviors, making it highly suitable for adaptive intrusion detection systems.

G. Model Comparison

To evaluate the effectiveness of different intrusion detection models, a comparative analysis was conducted using the Decision Tree, Random Forest, CNN,

LSTM, and Hybrid CNN-LSTM models. The performance of each model was measured using classification accuracy on the NSL-KDD testing dataset.



Accuracy Comparison of Different IDS Models

Model	Accuracy
Decision Tree	0.997023
Random Forest	0.998015
CNN	0.993570
LSTM	0.991427
Hybrid CNN-LSTM	0.992697

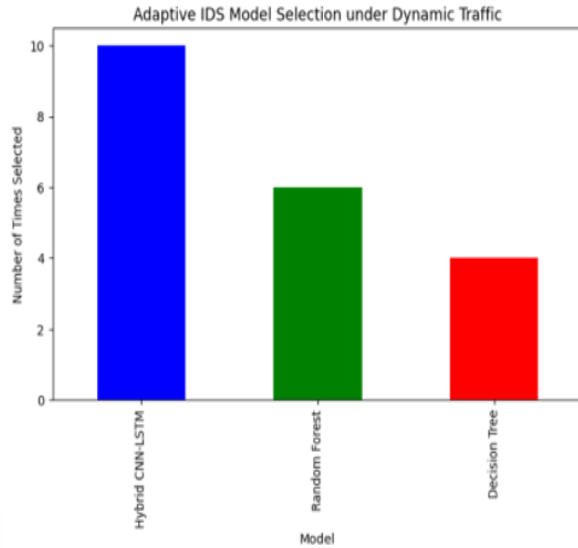
As it can be seen, all the classifiers have obtained quite high classification accuracy with values exceeding 99%. The classifier which showed the best accuracy is the Random Forest classifier, followed by the Decision Tree classifier. All deep learning-based models used, such as CNN, LSTM, and Hybrid CNN-LSTM, also exhibit great detection ability. Even though the Decision Tree and Random Forest classifiers provided better accuracy results compared to deep learning methods, these types of classifiers are capable of recognizing complex relationships in network traffic data. Therefore, they are suitable for detecting advanced multi-stage attacks. The comparison highlights that combining machine learning and deep learning tech-

niques within an adaptive intrusion detection framework can improve the robustness of the detection system.

H. Adaptive IDS Model Selection under Dynamic Traffic Conditions

To further evaluate the effectiveness of the proposed system, the Adaptive Intrusion Detection System (Adaptive IDS) was tested under simulated dynamic network traffic conditions. The system dynamically selects the most appropriate model based on the detected network traffic characteristics.

The below figure shows the frequency of model selection by the adaptive IDS during dynamic traffic simulations.



The results indicate that the Hybrid CNN-LSTM model was selected most frequently, particularly during attack-dominant traffic conditions. The Random Forest model was selected during mixed traffic scenarios, while the Decision Tree model was selected for normal traffic conditions. This adaptive selection mechanism enables the intrusion detection system to utilize the strengths of different models depending on the current network environment. As a result, the proposed adaptive IDS maintains high detection accuracy and improved robustness when operating under varying and dynamic network traffic conditions.

Confusion Matrix of Adaptive IDS

Actual/Pre-dicted	Normal	Attack
Normal	13452	17
Attack	33	11693

According to the confusion matrix, it is clear that there were 13,452 correct normal traffic predictions and 11,693 correct attack predictions. In addition, there were 17 incorrect normal traffic predictions and 33 incorrect attack predictions. It becomes evident from the above discussion that the proposed adaptive IDS has the ability to detect accurately with very low false positive and false negative error rates. This clearly shows that the adaptive model selection scheme is very effective in terms of enhancing the effectiveness of intrusion detection through the adoption of a proper model according to network traffic conditions.

Discussion

The results obtained from the experiments prove that the developed Traffic-Aware Adaptive Intrusion Detection System is able to detect intrusions efficiently using various machine learning and deep learning techniques. The conventional machine learning algorithms, like Decision Tree and Random Forest, performed extremely well in terms of classification accuracy and training speed. In addition, the deep learning algorithms, such as CNN, LSTM, and the hybrid CNN-LSTM model, effectively analyzed both spatial and temporal features of the network traffic data. Moreover, the adaptive algorithm for model selection ensured the robustness of the system because it was capable of choosing the best model according to network traffic characteristics.

V. CONCLUSION

In this research, Adaptive Intrusion Detection System (Adaptive IDS) was suggested to improve the security of the network when dealing with dynamic traffic in the network. The suggested system utilized both Machine Learning and Deep Learning methodologies such as Decision Tree, Random Forest, CNN, LSTM, and Hybrid CNN-LSTM. All these methods have been tested based on the NSL-KDD dataset, which is considered one of the most popular datasets used in intrusion detection studies. Among the evaluated models, the Random Forest and Decision Tree classifiers achieved the highest accuracy, while the deep learning

models showed strong capability in capturing complex patterns within network traffic data. The Hybrid CNN-LSTM model effectively combines spatial and temporal feature learning, enabling the system to detect more sophisticated intrusion patterns. In addition to improving the performance of the detection process, adaptive model selection was implemented. The experimental results confirm that the proposed Adaptive IDS achieves high detection accuracy, low false alarm rates, and improved robustness under dynamic network environments. Therefore, the proposed system can serve as an effective solution for modern intrusion detection systems and contribute to enhancing cybersecurity in real-world network environments.

VI. FUTURE SCOPE

Although the proposed Adaptive Intrusion Detection System (Adaptive IDS) demonstrates high accuracy and strong performance in detecting network intrusions, several improvements can be explored in future research. The first potential area of improvement concerns the evaluation process of the suggested IDS. Specifically, it can be improved by testing the algorithm on up-to-date benchmarking datasets. For example, the researchers may test the system on CICIDS2017 and UNSW-NB15 datasets, which have been recognized as benchmarks for intrusion detection studies. Second, the adaptive IDS of the paper relies on the previously set rules for the selection of detection models depending on specific traffic conditions. This problem might be addressed through implementing more sophisticated intelligent systems for determining the most appropriate detection method for particular network traffic conditions. Finally, the system can be deployed in a real-time network monitoring environment to evaluate its performance in practical cybersecurity applications. Integration with software-defined networking (SDN) or cloud-based security platforms could further enhance the scalability, adaptability, and real-time detection capabilities of the intrusion detection system.

VII. REFERENCES

[1] W. Liu, "Dynamic network intrusion detection model based on transformer and adversarial autoencoder," *International Journal of Intelligent Networks*, 2025, doi: 10.1016/j.ijin.2025.11.002.

[2] W. Villegas-Ch, J. Govea, R. Gutierrez, A. Maldonado Navarro, and A. Mera-Navarrete, "Effectiveness of an adaptive deep learning-based intrusion detection system," *IEEE Access*, vol. 12, pp. 184010–184023, 2024, doi: 10.1109/ACCESS.2024.3512363.

[3] U. Shahid, M. Z. Hussain, M. Z. Hasan, A. Haider, J. Ali, and J. Altaf, "Hybrid intrusion detection system for RPL IoT networks using machine learning and deep learning," *IEEE Access*, vol. 12, pp. 113099–113112, 2024, doi: 10.1109/ACCESS.2024.3442529.

[4] J. Du, K. Yang, Y. Hu, and L. Jiang, "NIDS-CNNLSTM: Network intrusion detection classification model based on deep learning," *IEEE Access*, 2023, doi: 10.1109/ACCESS.2023.3254915.

[5] H. C. Altunay and Z. Albayrak, "A hybrid CNN+LSTM-based intrusion detection system for industrial IoT networks," *Engineering Science and Technology, an International Journal*, vol. 38, Art. no. 101322, 2023, doi: 10.1016/j.jestch.2022.101322.

[6] I. A. Abdulmajeed and I. M. Husien, "MLIDS22—IDS design by applying hybrid CNN-LSTM model on mixed-datasets," *Informatica*, vol. 46, no. 8, pp. 121134, 2022, doi: 10.31449/Inf.v46i8.4348.

[7] Y. Wang, "Deep learning-based network intrusion detection systems," in *Proc. 2nd Int. Conf. Machine Learning and Automation*, 2024, doi: 10.54254/2755-2721/109/2024.18104.

[8] P. Sinha, D. Sahu, S. Prakash, T. Yang, R. S. Rathore, and V. K. Pandey, "A high performance hybrid LSTM-CNN secure architecture for IoT environments using deep learning," *Scientific Reports*, vol. 15, Art. no. 9684, 2025, doi: 10.1038/s41598-025-94500-5.

[9] S. Chatterjee, S. Chaudhary, and A. K. Cherukuri, "Intrusion detection system using deep learning for network security," *School of Computer Science Engineering and Information Systems, Vellore Institute of Technology, India*, 2024.

[10] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman,

- “Deep learning approach for intelligent intrusion detection system,” *IEEE Access*, vol. 7, pp. 41525–41550, 2019, doi: 10.1109/ACCESS.2019.2895334.
- [11] J. Du, K. Yang, Y. Hu, and L. Jiang, “A CNN-LSTM based intrusion detection model using KDD Cup99, NSL-KDD and UNSW-NB15 datasets,” *IEEE Access*, vol. 11, 2023, doi: 10.1109/ACCESS.2023.3254915.
- [12] O. Belarbi, T. Spyridopoulos, E. Anthi, I. Mavromatis, P. Carnelli, and A. Khan, “Federated deep learning for intrusion detection in IoT networks,” *arXiv preprint, arXiv:2306.02715*, 2023.
- [13] M. Cantone, C. Marrocco, and A. Bria, “Cross-da-ta-set evaluation of machine learning models for intrusion detection using CIC and LycoS datasets,” *IEEE Access*, vol. 11, 2023, doi: 10.1109/ACCESS.2024.3472907.
- [14] N. Xing, S. Zhao, Y. Wang, K. Ning, and X. Liu, “A dynamic intrusion detection system capable of detecting unknown attacks,” *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 14, no. 7, pp. 391–400, 2023.
- [15] J. Jabez, A. Alkhayyat, G. Vengatesan, and R. Vasanthan, “Adaptive intrusion detection mechanisms for enhancing security in cloud-hosted big data systems,” *Frontiers in Health Informatics*, vol. 13, no. 3, pp. 5313–5327, 2024.
- [16] A. A. Aliyu, J. Liu, and E. Gilliard, “A decentralized and self-adaptive intrusion detection approach using continuous learning and blockchain technology,” *Journal of Data Science and Intelligent Systems*, vol. 00, no. 00, pp. 111, Oct. 2024, doi: 10.47852/bonviewJDSIS42023803.
- [17] H. G. N. and A. H. S., “An enhanced network intrusion detection system using ADASYN and hybrid residual block techniques,” *Engineering, Technology & Applied Science Research*, vol. 15, no. 4, pp. 25082–25087, 2025.
- [18] M. Cantone, C. Marrocco, and A. Bria, “Machine learning in network intrusion detection: A cross-da-ta-set generalization study,” *IEEE Access*, vol. 11, pp. 121, 2024, doi: 10.1109/ACCESS.2024.3472907.
- [19] I. S. Thaseen and C. A. Kumar, “Intrusion detection model using fusion of chi-square feature selection and multi-class SVM,” *Journal of King Saud University– Computer and Information Sciences*, vol. 29, no. 4, pp. 462–472, 2017.
- [20] A. K. Cherukuri, S. T. Ikram, G. Li, and X. Liu, “Artificial intelligence-based approaches for anomaly detection,” in *Encrypted Network Traffic Analysis*. Cham, Switzerland: Springer, 2024, pp. 73–99.
- [21] V. D. Gowda, D. Palanikkumar, S. Dekka, K. D. V. Prasad, and S. Singh, “Future trends in V2X communication and interoperability,” in *Digital Convergence in Intelligent Mobility Systems*. John Wiley & Sons, Inc., Aug. 2025, pp. 217–239.
- [22] A. Kumar, B. Ashreetha, A. M. Reddy, S. Dekka, D. Gowda, and K. D. V. Prasad, “Maximizing energy efficiency in wireless sensor networks for IoT with advanced techniques and solutions,” in *Proceedings of the 5th International Conference on Pervasive Computing and Social Networking (ICPCSN)*. IEEE, May 2025, pp. 448–454.
- [23] S. Dekka, B. Sambana, K. N. Raju, D. M. Sai, M. Pallavi, and K. K. Reddy, “Performance evaluation of QoS in MAODV routing protocol in MANETS,” in *Advances in Machine Learning and Big Data Analytics I (ICMLBDA 2023)*, vol. 441. Springer Nature, Feb. 2025, pp. 133–150.
- [24] S. Dekka, K. N. Raju, D. M. Sai, M. Pallavi, and B. Sambana, “Minimize the energy consumption to increase the network lifetime for green IoT environment,” in *Advances in Machine Learning and Big Data Analytics I (ICMLBDA 2023)*, vol. 441. Springer Nature, Feb. 2025, pp. 151–159.
- [25] V. D. Gowda, M. S. P. A. Mary, A. Sharma, and S. Dekka, “Deep learning approaches for real-time data analytics in IoT sensor networks,” in *Universal Threats in Expert Applications and Solutions*, vol. 2. Springer Nature, Nov. 2024, pp. 239–248.